

# Итоговый отчет по проекту: Математическое моделирование прохождения шарика через изогнутую двумерную прямоугольную трубу

## 1. Постановка задачи

Целью проекта является разработка физически корректной двумерной модели поведения реального газа в коридоре произвольной формы под воздействием потенциальных сил.

В отличие от упрощенной модели идеального газа, в данной работе учтено межмолекулярное взаимодействие частиц, а также влияние внешних управляющих силовых полей (радиальных и линейных). Практическая задача симуляции — обеспечить доставку массива частиц из начальной зоны (спавн-области) через ломаный лабиринт коридоров в целевой резервуар за минимальное виртуальное время.

## 2. Основной цикл симуляции, расчет кинематики и проблема туннелирования

Движение частиц описывается классическими законами Ньютона.

Состояние каждой частицы  $i$  характеризуется декартовыми координатами  $\vec{r}_i = (x_i, y_i)$  и компонентами скорости  $\vec{v}_i = (v_{x,i}, v_{y,i})$ .

### Метод интегрирования уравнений движения

На каждом шаге симуляции для частицы вычисляется результирующее ускорение  $\vec{a}_i$ , создаваемое как силами парного межмолекулярного взаимодействия  $\vec{F}_{ij}$ , так и суперпозицией внешних полей:

$$\vec{a}_i = \frac{\sum_{j \neq i} \vec{F}_{ij}}{m_i} + \vec{a}_{\text{внешнее}}$$

На каждом шаге сначала считаем ускорение, затем обновляем скорость, а потом координату частицы, где новая координата рассчитывается уже по

пересчитанной скорости:

$$\vec{v}_i(t + dt) = \vec{v}_i(t) + \vec{a}_i(t) \cdot dt$$

$$\vec{r}_i(t + dt) = \vec{r}_i(t) + \vec{v}_i(t + dt) \cdot dt$$

Поскольку скорость изначально хранится в декартовых компонентах, алгоритм не требует постоянной дорогостоящей нормализации вектора скорости на каждом шаге; она применяется только при расчете тригонометрии отскоков от стен.

## Проблема крупного шага по времени ( $dt$ ) и численные артефакты

В первой версии симуляции при шаге  $dt = 0.01$  с проявились два опасных дефекта:

1. **Численный нагрев:** При крупном шаге  $dt$  частицы за один такт проскакивают глубоко в крутой потенциал отталкивания  $r-12$ , из-за чего алгоритм вычисляет аномально высокую силу и искусственно разгоняет их, нарушая закон сохранения энергии.
2. **Численное туннелирование:** Из-за искусственного разгона скорость частиц становилась настолько большой, что за один шаг частица физически «перепрыгивала» отрезок стены ( $v \cdot dt > \text{толщина коллизии}$ ).

## Решение: уменьшить $dt$

Для стабилизации системы шаг интегрирования  $dt$  был уменьшен в 10 раз — до 0.001 с. При этом на один графический кадр рендеринга выполняется ровно **10 микрошагов** вычисления физики. Дополнительно введено ограничение на максимальное ускорение. Это устранило туннелирование и обеспечило строгую энергетическую стабильность.

## 3. Потенциал Леннарда-Джонса и оптимизация радиуса отсечения

Для описания парного взаимодействия используется потенциал Леннарда-Джонса (6-12):

$$V(r) = 4\epsilon \left[ \left( \frac{\sigma}{r} \right)^{12} - \left( \frac{\sigma}{r} \right)^6 \right]$$

Сила взаимодействия  $\vec{F}_{ij} = -\nabla V(r)$  направлена вдоль линии центров частиц и равна:

$$\vec{F}_{ij} = \frac{24\epsilon}{r^2} \left( \frac{\sigma}{r} \right)^6 \left[ 2 \left( \frac{\sigma}{r} \right)^6 - 1 \right] \vec{r}_{ij}$$

- При  $r < 2^{1/6}\sigma \approx 1.122\sigma$  сила положительна (квантовое отталкивание электронных оболочек).
- При  $r > 2^{1/6}\sigma$  сила отрицательна (Ван-дер-Ваальсово дисперсионное притяжение).

## Оптимизация расчета потенциалов

Расчет парных сил имеет вычислительную сложность  $O(N^2)$ . Самой «дорогой» операцией для CPU является вычисление квадратного корня

`std::sqrt` при поиске расстояния  $r = \sqrt{r_x^2 + r_y^2}$ .

Для оптимизации было введено жесткое ограничение радиуса взаимодействия ( `cutoff = 15.0` ). Вызов корня вынесен под предварительную пре-чек проверку квадрата расстояния:

```
float cutoffSq = cutoff * cutoff;
if (rSq > cutoffSq) return {0.f, 0.f}; // Мгновенный выход без
вычисления корня

float r = std::sqrt(rSq); // Вычисляется только для гарантированно
близких частиц
```

Поскольку в силу разреженности газа большинство пар частиц удалены друг от друга, эта проверка позволила избежать вызова `std::sqrt` в ~90% случаев, дав чистый прирост производительности физического ядра на ~52%.

## 4. Внешние потенциальные поля

Для управления потоком газа используются внешние силовые поля двух типов. Для исключения физической сингулярности (деления на ноль при бесконечном сближении) в обоих полях применяется регуляризация ядра ( $d_{\text{eff}}$ ).

## 1. Радиальное поле (Точечный источник)

Моделирует поле точечного центра (аналог закона Кулона или Ньютона), спадающее по закону обратных квадратов ( $1/d^2$ ):

$$\vec{a}_{\text{radial}} = \frac{k_{\text{rad}}}{d_{\text{eff}}^3} \vec{d}$$

где  $d_{\text{eff}} = \max(d, d_{\text{min}})$ , а  $d_{\text{min}}$  — настраиваемый софт-потенциал, защищающий расчет от сингулярности в центре источника.

## 2. Линейное поле (Отрезок / Пластина)

Вычислительное упрощение (closest-point approximation), моделирующее поле бесконечной заряженной нити (закон обратного расстояния  $1/d$ ):

$$\vec{a}_{\text{segment}} = \frac{k_{\text{seg}}}{d_{\text{eff}}^2} \vec{d}$$

где  $\vec{d}$  — вектор от ближайшей точки на отрезке пластины до частицы, а  $d_{\text{eff}} = \max(d, 5.0)$  — фиксированный порог ограничения силы в непосредственной близости от обкладки. Аппроксимация считается за  $O(1)$  и заменяет взятие сложных интегралов поля отрезка конечной длины.

## 5. Геометрия системы, инициализация и алгоритмы пространственного анализа

### Чтение конфигурации и генерация по seed

Вся сцена (геометрия комнат, параметры частиц, зоны спавна и оптимизируемые поля) инициализируется через файл конфигурации `config.yaml`.

Для обеспечения строго воспроизводимого численного эксперимента генератор случайных чисел `std::mt19937` инициализируется фиксированным параметром `random_seed`. Это гарантирует, что при неизменных макропараметрах начальные координаты и векторы скоростей частиц генерируются абсолютно одинаково от прогона к прогону.

## Генерация стен и отсеков

Локация собирается из набора плоских многоугольников — отсеков (в коде — `compartments`). Чтобы частицы свободно перемещались между комнатами, программа на этапе загрузки собирает ребра всех полигонов. Если ребро одной комнаты геометрически совпадает с ребром другой, оно помечается как разделяемое (`shared`) и исключается из физических коллизий. Границами (стенами) становятся только неразделяемые внешние ребра.

При столкновении со стеной применяется метод **упругого отскока** скорости относительно нормали  $\vec{n}$ :

$$\vec{v}_{\text{после}} = \vec{v} - 2(\vec{v} \cdot \vec{n})\vec{n}$$

Это идеально сохраняет модуль скорости (кинетическую энергию) и гарантирует теплоизоляцию системы.

## Алгоритм Ray-Casting

Для определения триггерных зон (нахождения частицы на старте или финише) используется алгоритм трассировки лучей. Из координаты частицы  $\vec{p} = (x, y)$  проводится бесконечный горизонтальный луч вправо. Программа проверяет пересечение луча с ребрами полигона:

- Если координата  $y$  точки лежит между  $y$ -координатами концов ребра, вычисляется координата  $x$  геометрического пересечения луча с этим ребром.
- Если  $x$ -пересечения правее координаты  $x$  частицы — это истинное пересечение.

Нечетное число пересечений гарантирует, что точка находится **внутри** полигона.

```
if (((polygon[i].y > p.y) != (polygon[j].y > p.y)) &&
    (p.x < (polygon[j].x - polygon[i].x) * (p.y - polygon[i].y) /
     (polygon[j].y - polygon[i].y) + polygon[i].x))
{
    inside = !inside;
}
```

## 6. Глубокая оптимизация программного кода

Для проведения тысяч итераций автоматического поиска параметров кодовая база была оптимизирована для работы в высокоскоростном консольном ( `headless` ) режиме:

1. **AABB пре-чек коллизий со стенами:** Расчет точного расстояния до отрезков стен требует операций деления. Чтобы разгрузить процессор, была внедрена предварительная грубая отсечка по ограничивающей рамке (Axis-Aligned Bounding Box) с учетом радиуса частицы:

```
float minX = std::min(a.x, b.x) - radius; // ... аналогично для
maxX, minY, maxY
if (pos.x < minX || pos.x > maxX || pos.y < minY || pos.y >
    maxY) return;
```

Благодаря этому все далекие стены отбрасываются мгновенно элементарными операциями сравнения.

2. **Флаги компиляции:** В `CMakeLists.txt` были включены агрессивные флаги компилятора.

**Результат оптимизации:** Физический движок ускорился **более чем в 20 раз**. Прогон симуляции до таймаута (1000 виртуальных секунд) сократился со 120 секунд до **22 секунд** реального времени.

## 7. Исследовательская часть: Автоматический поиск параметров методом Дифференциальной эволюции

Задача поиска оптимальных интенсивностей полей  $\vec{k}^* = (k_1, k_2, \dots, k_n)$  для эффективного проведения газа через повороты является задачей оптимизации «черного ящика» (black-box) с высокой зашумленностью и стохастической природой. Для ее решения был интегрирован эволюционный метод `scipy.optimize.differential_evolution`.

### Алгоритм работы оптимизатора:

Алгоритм инициализирует популяцию из 20 векторов интенсивностей и итеративно улучшает каждый из них:

1. **Выбор родителя:** Берется вектор №1 (Родитель) для попытки улучшения.
2. **Выбор секундантов:** Из популяции случайно выбираются три *других* вектора (например,  $\vec{k}_A, \vec{k}_B, \vec{k}_C$ ).
3. **Мутация:** На основе них строится вектор-мутант:  $\vec{v} = \vec{k}_A + F \cdot (\vec{k}_B - \vec{k}_C)$ .
4. **Скращивание:** Компоненты Родителя №1 и Мутанта случайным образом перемешиваются, порождая итогового **Потомка**.
5. **Оценка и селекция (Вызов C++ ядра):** Для параметров Потомка запускается одиночная симуляция в `headless`-режиме. Если Потомок показывает штрафную стоимость (cost) меньше, чем у Родителя №1, он занимает его место в таблице популяции. Иначе — выбрасывается.

### Целевая функция (Метрика штрафов)

В MVP-версии оптимизатора была заложена комбинированная метрика. Рабочим критерием успешности для оптимизатора является прохождение **80% частиц** (хотя полная симуляция в C++ считает абсолютным успехом только 100%).

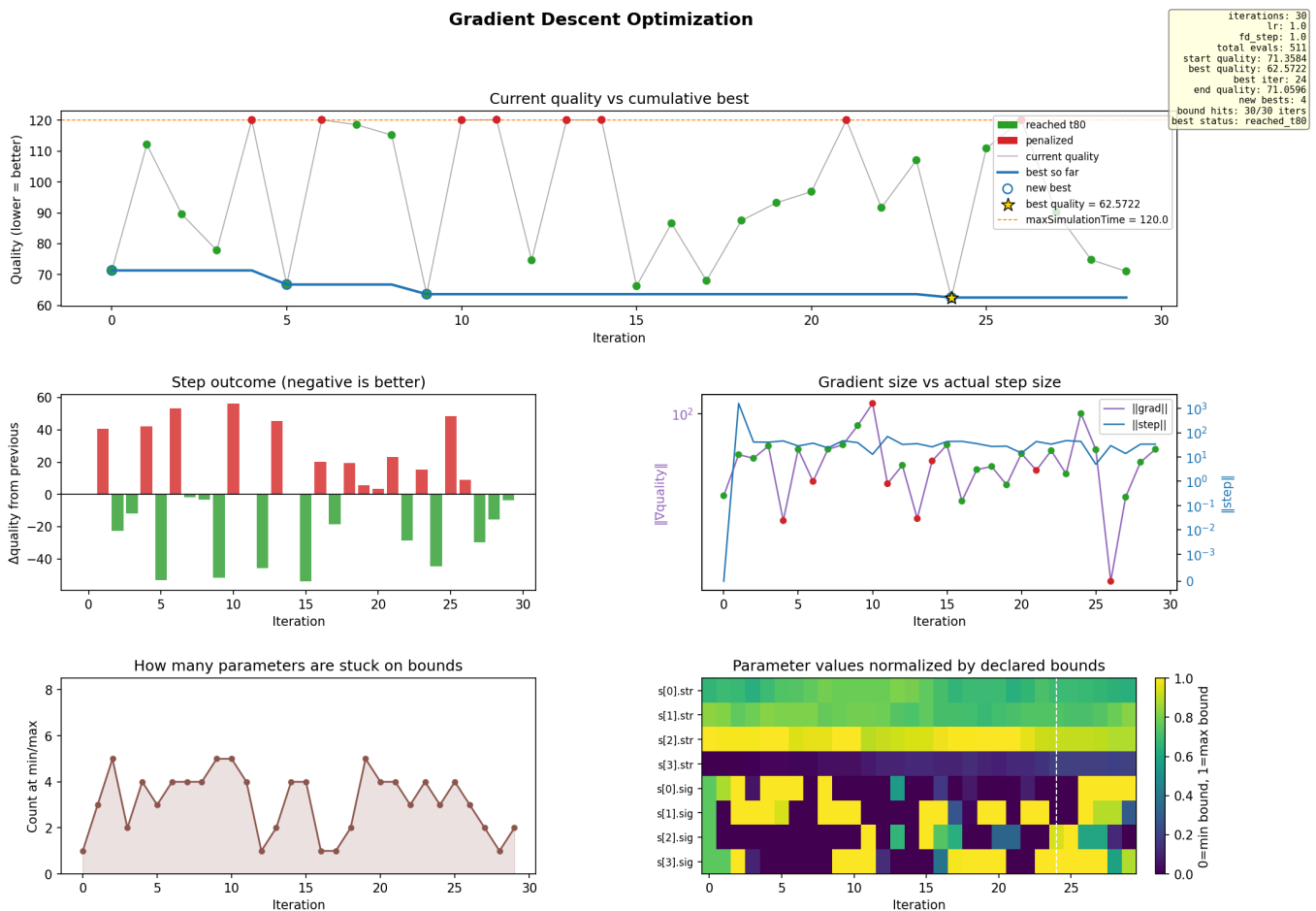
- Если порог в 80% достигнут:  $J(\vec{k}) = t_{\text{virtual}}$  (виртуальное время прохождения, поощряются быстрые конфигурации).

- Если порог не достигнут, применяется жесткий штраф:

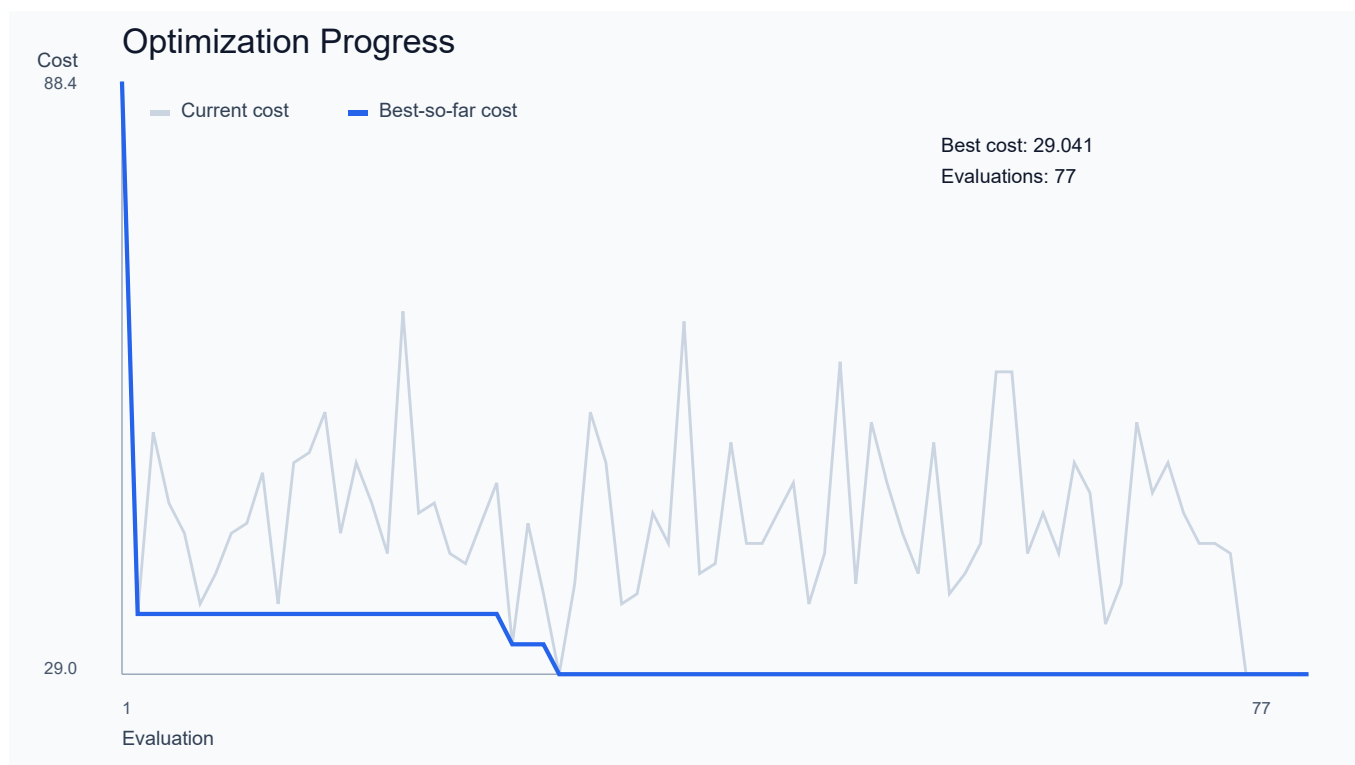
$J(\vec{k}) = 1000 + (N_{\text{target}} - N_{\text{reached}}) \cdot 100$  (чем больше частиц застряло в мертвых зонах, тем хуже штраф).

## 8. Сравнение Дифференциальной эволюции с Градиентным спуском

В рамках исследования был проведен сравнительный анализ применимости Дифференциальной эволюции (DE) и классического Градиентного спуска (GD) для данной физической системы:



Шумный график в разных вариациях оптимизации GD для старой версии полей (6 параметров)



наглядный график оптимизации DE для новой версии (4 параметра)

| Критерий сравнения                  | Дифференциальная эволюция (DE)                                                            | Градиентный спуск (GD)                                                           |
|-------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Требование к градиенту              | Не требуется (Zero-order метод).                                                          | Требуется вычисления градиента $\nabla J(\vec{k})$ (аналитически или численно).  |
| Поведение на зашумленных ландшафтах | Великолепное. Игнорирует мелкие стохастические флуктуации за счет стохастического поиска. | Крайне слабое. Застывает в первой же микро-яме, вызванной случайным seed частиц. |
| Глобальный vs Локальный поиск       | Глобальный. Работает с популяцией, способен перепрыгивать локальные экстремумы.           | Локальный. Находит только ближайший локальный минимум от начальной точки.        |
| Аналитическая структура             | Рассматривает систему как "черный ящик".                                                  | Требуется гладкости и непрерывности целевой функции.                             |

**Вывод:** Градиентный спуск в данной задаче принципиально несостоятелен. Поскольку траектории частиц имеют хаотический характер (из-за нелинейности Леннарда-Джонса), функция качества  $J(\vec{k})$  является негладкой, разрывной и "шумной". Градиентный спуск мгновенно застревает в локальных флуктуациях, в то время как Дифференциальная эволюция успешно находит глобальный баланс сил в каналах.

## 9. Физический смысл модели и практическое применение

### 1. Управление пучками заряженных частиц (ионные двигатели и датчики)

Если представить, что наши частицы — это не просто газ, а **заряженные ионы** (например, атомы аргона, у которых оторвали электрон), то симуляция превращается в модель реального прибора:

- **То, как частицы отталкиваются друг от друга** в упор, похоже на то, как расталкиваются в разные стороны одинаково заряженные ионы в физике.
- **Длинные плоские поля (пластины)** работают как электроды-фокусирующие. Они сжимают поток ионов с боков, не давая ему разлетаться.
- **Точечные поля (радиальные)** работают как невидимые магнитные или электрические линзы. Они могут собирать пучок в одну точку или поворачивать его.

Именно так ученые управляют потоком частиц в космических **ионных двигателях** или в **масс-спектрометрах** (приборах, которые определяют состав сложного вещества).

### 2. Лазерные "пинцеты" и микроскопические капли

В физике жидких и мягких веществ наша модель описывает технологию **оптического пинцета**:

- **Точечные притягивающие поля** в симуляции ведут себя точно так же, как **сфокусированный луч лазера**. Если направить лазер определенным образом, он может "схватить" и удерживать на месте микроскопическую пластиковую сферу или бактерию, не повреждая ее.
- С помощью этой модели можно изучать, как такие крошечные частицы ведут себя внутри микроскопических каналов с жидкостью, когда на них светят лазером и они сталкиваются друг с другом, словно упругие мячики.

## 10. Направления дальнейшего развития модели

От чего мы сознательно отказались ради MVP оптимизатора и простоты реализации:

- Не стали усреднять качество по многим seed.  
Вместо этого фиксируем один `random_seed`, чтобы сравнение кандидатов было воспроизводимым и код оставался простым.
  - Не стали оптимизировать геометрию сцены.  
Оптимизатор меняет только интенсивности уже существующих `segment_fields`, а не добавляет новые поля, не двигает их и не меняет форму коридоров.
  - Не стали делать общую сложную систему параметров для всех объектов.  
В MVP оптимизируются только поля, у которых явно стоит `optimize: true`.
  - Не стали строить сложную многофакторную целевую функцию.  
Взяли простую практическую метрику: как быстро достигается целевой процент частиц, плюс штраф, если он не достигнут.
  - Коридор можно строить не численно через координаты в конфиг-файле, а через утилиты для геймдева, например `Tiled`
  - Для универсального поиска поля по любому коридору, можно уже написать свою маленькую нейросеть и обучить ее искать наилучшие поля